

ZLC FPGA Pulse Streamer 测试报告

40-channel entryptpoint / GUI subset compile / server contract

Vivado 2019.x、runtime edge table、Pulse GUI 与 sequencer server 的
当前验证摘要

2026-06-03

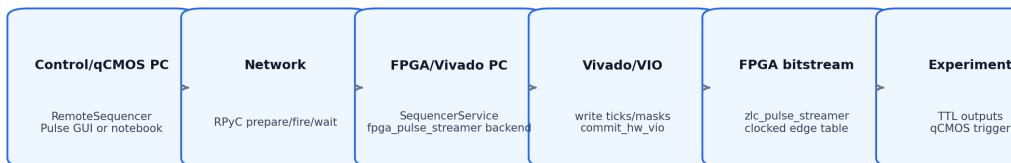
Contents

1	结论先行	1
2	入口和目录	3
3	40-channel 编译契约	4
4	Camera Imaging Preset	5
5	验证命令	6

1

结论先行

本报告记录当前仓库中 FPGA pulse-streamer 的 40-channel 入口、Pulse GUI/API 编译边界、server 启动边界和文档同步状态。当前硬件路线是固定的 40 路 runtime pulse-streamer: GUI 可以只显示或配置少数 channel, 但 server 上传给 FPGA 的 program 始终按 `ch00 ... ch39` 补齐, 未使用 channel 的 mask bit 为 0。



Only upload/start timing is host dependent. Pulse edges after start are executed by the FPGA clock.

Figure 1.1: 控制电脑、FPGA server、Vivado/VIO、FPGA bitstream 和 qCMOS trigger 的边界。GUI 在前端; 40 路补齐发生在 sequencer compile/upload 边界。

本轮重点检查:

- `fpga\build_and_program.bat` 是 40 路 build/program/check/diagnose 的统一入口。
- `fpga\run_server.bat` 永远启动 `ch00 ... ch39` 的 40 路 server, trigger 为 `ch03`。
- `fpga\pulse_streamer` 下旧的分散 Windows bat 已删除, Vivado build 产物目录已从源码树清理。
- `pulses\camera_imaging_40ch.json` 默认只显示前四路, 但保存和编译仍保留完整 40 路硬件顺序。

-
- 只包含 `ch00/ch03` 的 GUI/API state 编译到 40 路硬件时, `program.channels` 是完整 40 路, 所有高于 `ch03` 的 mask bit 都为 0。

2

入口和目录

从 repo 根目录运行:

Command line

```
.\fpga\build_and_program.bat --help
.\fpga\build_and_program.bat --check
.\fpga\build_and_program.bat
.\fpga\run_server.bat
```

`-check` 做无 XDC 的 40 路 synth 自查; 真实 build/program 需要完成 `fpga\pulse_streamer\zlc_pulse_streamer_40ch.xdc`, 或设置 `ZLC_PS_40CH_XDC` 指向完成的板级 XDC。脚本会拒绝仍包含 `<PIN_CHxx>` placeholder 的 XDC。

`pulse_gui.bat` 仍在 repo 根目录, 因为它是前端入口, 不是 FPGA build/server 脚本:

Command line

```
.\pulse_gui.bat --remote-host 127.0.0.1 --remote-port 18861 --state
↪ .\pulses\camera_imaging_40ch.json
```

3

40-channel 编译契约

`PulseTableState` 可以包含完整 40 路，也可以只包含当前使用的硬件 channel 子集。真正上传前，`compile_pulse_table_runtime_program(state, channels=hardware_channels)` 使用 server 的完整硬件 channel list 作为 bit order:

Python

```
hardware_channels = [f"ch{i:02d}" for i in range(40)]
state = na.PulseTableState(
    channels=["ch00", "ch03"],
    periods=[
        na.PulsePeriod(10, (1, 0), unit="ns"),
        na.PulsePeriod(10, (0, 1), unit="ns"),
        na.PulsePeriod(10, (0, 0), unit="ns"),
    ],
    visible_channels=["ch00", "ch03"],
)
program = na.compile_pulse_table_runtime_program(
    state,
    channels=hardware_channels,
    clock_hz=100_000_000,
    trigger_channels=["ch03"],
)
```

预期结果:

Expected

```
program.channels = ["ch00", ..., "ch39"]
program.masks = [1 << 0, 1 << 3, 0, 0]
all(mask >> 4 == 0 for mask in program.masks)
program.repeat_forever is True
```

这就是 “GUI 只显示少数 channel, FPGA 仍然正常跑 40 路” 的边界。Verilog 不需要知道 GUI 隐藏了哪些行；它只接收已经补齐的 40-bit mask。

4

Camera Imaging Preset

`pulses\camera_imaging_40ch.json` 是当前相机成像 preset。它的硬件 channel 是 `ch00...ch39`，默认 visible channel 是 `ch00/ch01/ch02/ch03`，display label 分别是 `trap/cooling/probe/qcm_trigger`。

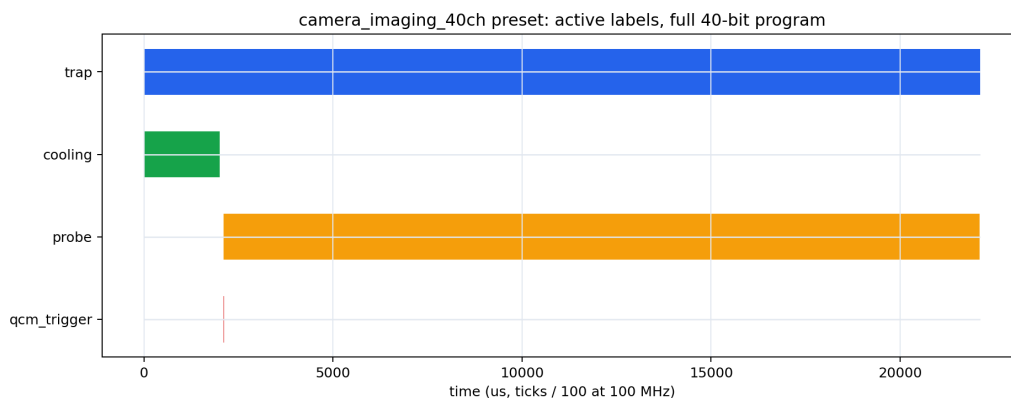


Figure 4.1: 40-channel camera imaging preset 的 edge table。图中只使用前四路，上传 program 仍然带完整 40 路 channel list。

100 MHz 下这个 preset 编译为：

camera imaging 40ch

```
ticks: 0, 200000, 210000, 212000, 2210000, 2212000
masks: 3, 1, 13, 5, 1, 0
trigger_count: 1
repeat_forever: true
```

5

验证命令

本轮相关验证命令：

Command line

```
cmd /c fpga\build_and_program.bat --help
cmd /c fpga\run_server.bat --help
python -m json.tool tutorials\nneutral_atom_fpga_server.ipynb
python -m json.tool tutorials\nneutral_atom_hardware_quickstart.ipynb
pytest -q tests\test_neutral_atom_lightweight.py
↪ tests\test_frontend_smoke.py
```

如果在真实 FPGA 电脑上继续验证，推荐顺序是：

1. 把 repo 放到短路径，例如 `D:\ZLC`。
2. 设置 `ZLC_PS_VIVADO_BIN`，如果 Vivado 不在默认搜索路径。
3. 填完 `zlc_pulse_streamer_40ch.xdc`。
4. 运行 `fpga\build_and_program.bat -check`。
5. 运行 `fpga\build_and_program.bat` 生成并 program 40 路 bitstream。
6. 运行 `fpga\run_server.bat`。
7. 从 FPGA 电脑或控制电脑运行 `pulse_gui.bat -remote-host ... -state .\pulses\camera_imaging_40ch.json`。